Лабораторная работа №9. Переменные окружения и конфигурация контейнеров

Тема: Конфигурирование Nginx, Express API и PostgreSQL через .env, environment и env_file

Цель: Научиться задавать и изменять параметры Docker Compose без редактирования исходного кода, проверять окружение контейнеров и диагностировать ошибки подключения.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №9. Переменные окружения и конфигурация контейнеров. Продолжительность: 4 академических часа (180 минут). Среда: Debian 13, Docker Engine, Docker Compose, Express.js, Nginx, PostgreSQL 17. Продолжение практического проекта №8.

Цель: разделить исходный код и конфигурацию многоконтейнерного приложения, применить .env, env_file и environment, проверить изменения порта и параметры подключения без пересборки Node.js и без потери данных.

2. Требования к подготовке

- Доступ в Debian 13 с установленным Docker Engine и Compose plugin.
- Рабочий проект лабораторной №8 compose-course8, содержащий API /api/products, Dockerfile, nginx/default.conf и db/init.sql.
- Свободен порт 8080 на Debian; если нет, используйте другой учебный HTTP_PORT.
- Для команд управления Docker используйте sudo, если ваша учётная запись не имеет доступа к Docker socket.

Во всех примерах команды docker compose приведены без sudo; при необходимости добавляйте sudo. Выполняйте действия только на учебной виртуальной машине. Не размещайте реальный пароль в открытом репозитории.

3. Краткая теория

Основные механизмы

Механизм	Роль
.env	Значения для интерполяции compose.yaml
environment	Переменные в окружении конкретного сервиса
env_file	Подключение файла переменных к сервису
ARG	Параметр Dockerfile во время сборки
ENV	Значение переменной, заданное Dockerfile для образа/контейнера

Важно: один только .env не передаёт переменные в контейнер; для этого используют environment и/или env_file. Эти механизмы не шифруют секреты.

4. Задание 1. Создание независимой копии проекта

```
Создайте копию проекта №8
```

```
cp -a ~/compose-course8 ~/compose-course9
cd ~/compose-course9
ls -la
```

Если каталог находится в другом месте, подставьте фактический путь. Не копируйте каталог с именованными Docker Volumes: томами управляет Docker отдельно от рабочего каталога. Новый каталог Compose обычно образует отдельный проект и собственный том.

Проверка состава проекта

```
find . -maxdepth 3 -type f | sort
```

Убедитесь, что существуют api/config/db.js, api/Dockerfile, nginx/default.conf, db/init.sql и compose.yaml. Если какого-либо файла нет, получите его из материалов лабораторной №8.

5. Задание 2. Файл .env для Compose

Создайте файл .env

```
HTTP_PORT=8080
DB_NAME=shop
DB_USER=shopuser
DB_PASSWORD=training_only_change_me
```

Внимание: в предыдущей работе мог использоваться другой пароль. Это отдельный проект с новым томом; здесь задаём демонстрационный пароль до первой инициализации PostgreSQL. На настоящем сервере не используют учебные значения.

Создайте файл .env.example

```
HTTP_PORT=8080
DB_NAME=shop
DB_USER=shopuser
DB_PASSWORD=SET_YOUR_OWN_PASSWORD
```

Добавьте правила в .gitignore

```
.env
node_modules/
.env.*
!.env.example
```

Правило .gitignore препятствует случайному добавлению новых файлов .env в Git, но не защищает содержимое уже опубликованных секретов.

6. Задание 3. Файл api.env

Создайте api.env в корне проекта

```
DB_HOST=db
DB_PORT=5432
APP_LABEL=compose-course9
```

Файл api.env не содержит пароль. В нём три настройки окружения, которые будут доступны только сервису api посредством env_file.

7. Задание 4. Изменение compose.yaml

Полный compose.yaml

```
services:
  nginx:
    image: nginx:stable
    ports:
      - '127.0.0.1:${HTTP_PORT:-8080}:80'
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - api

  api:
    build: ./api
    env_file:
      - ./api.env
    environment:
      DB_NAME: ${DB_NAME:?DB_NAME is required}
      DB_USER: ${DB_USER:?DB_USER is required}
      DB_PASSWORD: ${DB_PASSWORD:?DB_PASSWORD is required}
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:17
    environment:
      POSTGRES_DB: ${DB_NAME:?DB_NAME is required}
      POSTGRES_USER: ${DB_USER:?DB_USER is required}
      POSTGRES_PASSWORD: ${DB_PASSWORD:?DB_PASSWORD is required}
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U "$$POSTGRES_USER" -d "$$POSTGRES_DB"']
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  pgdata:
```

Пояснение: ports публикует Nginx только на loopback-интерфейсе Debian; другие сервисы доступны через внутреннюю сеть Compose. DB_HOST=db поступает в api из api.env, а имя базы, пользователь и пароль — через environment. Двойной знак $$ предотвращает интерполяцию переменной Compose в строке healthcheck и оставляет её для оболочки внутри контейнера.

Файлы api/app.js, Router, Controller, Service, api/config/db.js, nginx/default.conf и db/init.sql оставьте из лабораторной №8 без переписывания: конфигурация db.js уже читает DB_* через process.env.

8. Задание 5. Проверка конфигурации и запуск

```
Проверка без вывода паролей

docker compose config -q
docker compose config --services
docker compose config --volumes
```

docker compose config показывает развёрнутую конфигурацию, включая пароли: его полный вывод не прикладывайте к отчёту. Команда -q позволяет проверить синтаксис без вывода настроек.

```
Первый запуск

docker compose up -d --build
docker compose ps
```

Дождитесь состояния healthy для db. Проверьте, что Nginx доступен через порт, определённый HTTP_PORT.

```
Проверка REST API

curl -i http://127.0.0.1:8080/api/products
```

Ожидается HTTP 200 и JSON-массив товаров. Если в новой БД нет записей, пустой массив [] допустим.

9. Задание 6. Проверка среды контейнера

```
Просмотр несекретных значений из Node.js

docker compose exec api node -e
"console.log({DB_HOST:process.env.DB_HOST,DB_PORT:process.env.DB_PORT,DB_NAME:process.env.DB
_NAME,APP_LABEL:process.env.APP_LABEL,passwordSet:Boolean(process.env.DB_PASSWORD)})"
```

Зафиксируйте только DB_HOST=db, DB_PORT=5432, DB_NAME=shop, APP_LABEL=compose-course9 и passwordSet=true. Значение DB_PASSWORD не выводите.

10. Задание 7. Смена опубликованного порта

1. Измените HTTP_PORT=8080 на HTTP_PORT=8081 в .env.
2. Проверьте конфигурацию через docker compose config -q.
3. Выполните docker compose up -d: Compose должен применить изменённую конфигурацию сервиса nginx.
4. Откройте API по новому адресу и подтвердите, что старый порт не используется данным проектом.
5. Верните HTTP_PORT=8080 и выполните docker compose up -d.

```
Проверка порта

curl -i http://127.0.0.1:8081/api/products
docker compose ps
```

Смена порта хоста не требует пересборки образа Node.js. Если порт 8081 занят, выберите свободный порт и отразите его в отчёте.

11. Задание 8. Диагностический эксперимент

Намеренно нарушьте подключение API к БД, заменив DB_HOST=db на DB_HOST=db-wrong в файле api.env. Выполните команды ниже, запишите наблюдения, затем обязательно восстановите конфигурацию.

Воспроизведение ошибки

```
docker compose up -d --force-recreate api
curl -i http://127.0.0.1:8080/api/products
docker compose logs --tail=30 api
```

Ожидается ошибка DNS или подключения к базе данных; конкретный статус HTTP зависит от обработки ошибок в API лабораторной №8.

Обязательное восстановление

```
sed -i "s/^DB_HOST=db-wrong$/DB_HOST=db/" api.env
docker compose up -d --force-recreate api
curl -i http://127.0.0.1:8080/api/products
```

Убедитесь, что запрос после восстановления снова работает; файл api.env должен содержать DB_HOST=db.

12. Задание 9. Сохранность данных PostgreSQL

Создание тестовой записи

```
curl -i -X POST http://127.0.0.1:8080/api/products \
  -H 'Content-Type: application/json' \
  -d '{"name":"Environment Demo","price":990}'
```

Перезапуск проекта без удаления тома

```
docker compose down
docker compose up -d
curl -i http://127.0.0.1:8080/api/products
```

Проверьте, что добавленная запись сохранилась. Не выполняйте docker compose down -v: эта команда удаляет именованный том проекта.

13. Самостоятельное задание

6. Самостоятельно добавьте в api.env переменную COURSE_GROUP со значением своей учебной группы (например, IS-21).
7. Пересоздайте только сервис api.
8. Выведите COURSE_GROUP через docker compose exec api node -e без изменения исходного кода.
9. Поясните, почему переменная не появилась в уже выполнявшемся процессе до пересоздания контейнера.

14. Отчёт

10. Тема, цель, исходная структура проекта и используемое окружение.
11. Фрагменты .env.example, api.env и compose.yaml без реального пароля.
12. Скриншоты docker compose ps, результата GET, переменных DB_HOST/DB_PORT, смены HTTP_PORT.
13. Описание преднамеренной ошибки DB_HOST и последовательности восстановления.
14. Подтверждение сохранности тестового товара после down/up.
15. Вывод о различиях .env, environment и env_file.

15. Контрольные вопросы

16. Какую задачу выполняет файл .env для Compose?
17. Почему значения из .env не обязаны оказаться в контейнере?
18. Чем отличаются environment и env_file?
19. Почему DB_HOST=db, а не localhost?
20. Зачем используется ${HTTP_PORT:-8080}?
21. Почему опасно прикладывать вывод docker compose config целиком?
22. Чем ARG отличается от ENV?
23. Почему смена .env может требовать пересоздания контейнера?
24. Почему изменение POSTGRES_PASSWORD в .env не меняет пароль существующей роли в уже инициализированной БД?
25. Чем down отличается от down -v?

16. Критерии оценивания

Оценивание

Критерий	Баллы
Корректные .env, .env.example и api.env	15
Изменённый compose.yaml и проверка config -q	25
Успешный GET через Nginx и просмотр переменных api	15
Переключение HTTP_PORT и восстановление	15
Диагностика DB_HOST и восстановление связи	15
Сохранность данных и отчёт	10
Самостоятельная переменная COURSE_GROUP	5
Итого	100

17. Итог

В результате лабораторной студенты разделили образ приложения и конфигурацию окружения, научились менять параметры Compose, проверять переменные внутри контейнера и восстанавливать работу системы после ошибки конфигурации. Данные PostgreSQL сохранены в томе.